

PSYC 51.17: Models of Language and Communication

Lecture 17: Training¹ transformers

**Week 5, Lecture 3 - From Architecture to
Implementation**

PSYC 51.17: Models of Language and Communication

Winter 2026

Winter 2026

Today's Agenda

1. **Positional Encoding:** Injecting sequence order
2. **Feed-Forward Networks:** The other key component
3. **Layer Norm & Residuals:** Training deep networks
4. **FlashAttention:** Making transformers faster
5. **Three Architectures:** Encoder, Decoder, Both
6. **Practical Implementation:** Using HuggingFace

*Goal: Complete understanding of transformer training
and implementation*

Winter 2026

Positional encoding

Problem: Self-attention is permutation-invariant!

Without position information:

- "The cat sat" = "sat cat the" = "cat the sat"
- Order matters in language!

Solution: Add positional encodings to input embeddings

Why sinusoidal positional encoding?

Advantages of sine/cosine functions:

1. Unique Patterns

- Each position gets unique vector
- Different frequencies per dimension

Visualization:

1	Dim 0 (high freq): ~~~~~ (fast oscillation)	4
2	Dim 1 (mid freq): ~~~	

2. Relative Position Info

Why sinusoidal positional encoding?

11 | Each position has a unique "barcode"!

Winter 2026

Visualizing positional encodings

Each position gets a unique pattern across dimensions

1 | Pos 0 → Pos 5 → Pos 9 → Dim 0 → Dim 4 → Dim 7

Key Insight:

- Lower dimensions: Rapid oscillation (high frequency)
- Higher dimensions: Slow oscillation (low frequency)
- Creates a unique "barcode" for each position
- Model learns to use these patterns for positional awareness

Feed-forward networks

After attention, apply position-wise feed-forward network

Architecture: Two linear layers with ReLU/GELU activation

```
1 | class FeedForward(nn.Module):
```

Purpose: Add non-linearity and increase model capacity

- Attention is mostly linear (weighted sums)
- FFN adds expressiveness through ReLU/GELU

2. Position-wise Processing

- Attention mixes information across positions
- FFN processes each position independently
- Allows position-specific feature transformations

3. Increase Model Capacity

- Expansion (4x) provides more parameters
- Can learn complex feature combinations

Layer normalization & residual connections

Critical for training deep transformers!

Residual Connections:

```
1 # output =  
  sublayer(x) + x  
2 x = x +  
  self.attention(x)  
3 x = x +  
  self.feedforward(x)
```

Layer Normalization:

```
1 # Normalize  
  across  
  features (not  
  batch)  
2 def  
  layer_norm(x)
```

Pre-norm vs post-norm

Two ways to arrange LayerNorm and residual connections

Post-Norm (Original):

1	# Normalize AFTER residual
2	$x =$ $\text{LayerNorm}(x +$

Pre-Norm (Modern):

1	# Normalize BEFORE sublayer
2	$x = x +$ $\text{Attention}(\text{LayerNorm}$
3	$x = x +$

Complete transformer block

Putting it all together:

```
1 | class TransformerBlock(nn.Module):
```

Model sizes:

- BERT-base: 12 blocks, 110M params | BERT-large: 24 blocks, 340M params
- GPT-3: 96 blocks, 175B params!

Winter 2026

FlashAttention: Making transformers faster

Problem: Standard attention is slow and memory-hungry!

Standard Attention Complexity

- Time: $O(n^2)$ where n = sequence length - Memory: $O(n^2)$ to store attention matrix - Bottleneck: Reading/writing to GPU memory (HBM)

FlashAttention Innovation:

- Tile-based computation
- Uses fast SRAM instead of slow HBM
- Fused operations (fewer memory reads)
- Recomputation in backward pass

Concrete Speedup:

Method	Speed	Memory
Standard	1x	1x
FlashAttention	3x faster	0.5x

Real Impact:

- Sequence 1024→4096 on same GPU
- Training 2-4x faster
- Used in: LLaMA, GPT-4, Mistral

```
1 # Standard: materialize full n×n
  matrix
2 attn = softmax(Q @ K.T /
```

- Approximate attention with linear complexity
- Kernel trick to avoid materializing attention matrix
- Used in: Performer, Linear Transformer

3. Low-Rank Approximation

- Factorize attention matrix
- Reduce memory footprint

Three transformer architectures

Architecture	How it works	Examples	Uses
Encoder-Only	Full self-attention	BERT, RoBERTa	Classification, NER, QA
Decoder-Only	Causal/masked attention	GPT-2, GPT-3	Generation, completion
Encoder-Decoder	Full self-attention	Seq2Seq, BART	Machine translation, summarization

Visual comparison: Encoder vs Decoder vs Both

Encoder-Only (BERT): Self-Attention → bidirectional

Decoder-Only (GPT): Masked Attention → causal

Choosing the Right Architecture:

- Need to understand full context? → **Encoder** (BERT)
- Need to generate text? → **Decoder** (GPT)
- Need both (translate, summarize)? → **Encoder-Decoder** (T5)

Using transformers in practice

HuggingFace makes it easy!

```
1 from transformers import AutoModel, AutoTokenizer
2
3 # Load pre-trained model (downloads ~440MB first time)
4 model_name = "bert-base-uncased"
5 tokenizer = AutoTokenizer.from_pretrained(model_name)
6 model = AutoModel.from_pretrained(model_name)
7
8 # Tokenize input
9 text = "The animal didn't cross the street because it was too tired"
10 inputs = tokenizer(text, return_tensors="pt")
11 print(inputs['input_ids'])
12 # tensor([[ 101, 1996, 4111, 2134, 1005, 1056, 2892, 1996,
13 # 2395, 2138, 2009, 2001, 2205, 5458, 102]])
14 # [CLS] The animal didn ' t cross the ...
15
16 # Get contextualized embeddings
```

16

Winter 2026

continued...

Using transformers in practice

```
17 outputs = model(**inputs)
18 hidden_states = outputs.last_hidden_state # [1, 15, 768]
19
20 # "it" is at position 10 - its embedding knows it refers to "animal"!
21 it_embedding = hidden_states[0, 10, :] # 768-dim context-aware vector
```

Reference: HuggingFace Course - Chapter 1.4

Winter 2026

Comparing architectures: Code examples

Different architectures for different tasks

```
1 from transformers import AutoModelForSequenceClassification, \
2   AutoModelForCausalLM, \
3   AutoModelForSeq2SeqLM
4
5 # 1. ENCODER-ONLY (BERT): Classification
6 encoder_model = AutoModelForSequenceClassification.from_pretrained(
7   "bert-base-uncased", num_labels=2
8 )
9 # Use for: Sentiment analysis, NER, classification
10
11 # 2. DECODER-ONLY (GPT): Text generation
12 decoder_model = AutoModelForCausalLM.from_pretrained("gpt2")
13 # Use for: Text completion, creative writing, few-shot learning
14
15 # 3. ENCODER-DECODER (T5): Seq2Seq tasks
16 seq2seq_model = AutoModelForSeq2SeqLM.from_pretrained("t5-base")
17 # Use for: Translation, summarization, question answering
```

18

Winter 2026

continued...

Comparing architectures: Code examples

18

19 # All use transformer architecture, different attention patterns!

Key Takeaway: Choose architecture based on your task!

Winter 2026

Practical tips for training transformers (Part 1)

1. Learning Rate & Warmup

```
1 # Warmup: gradually increase LR
2 # Then decay (linear or cosine)
3 scheduler = get_linear_schedule_with_warmup(
4     optimizer,
```

2. Optimizer

```
1 optimizer = AdamW(
2     model.parameters(),
3     lr=2e-5.
```

Winter 2026

Practical tips for training transformers (Part 2)

3. Regularization

```
1 # Dropout after attention and FFN
2 self.dropout = nn.Dropout(0.1)
3
4 # Gradient clipping
```

4. Mixed Precision (FP16)

```
1 from torch.cuda.amp import autocast
2
3 with autocast(): # Use FP16
```

1. Batch Size

- Larger batches = better GPU utilization
- Use gradient accumulation if GPU memory limited
- Typical: effective batch size 256-2048 tokens

2. Sequence Length

- Shorter sequences train faster (quadratic complexity!)
- Consider truncation or sliding windows
- Pack multiple examples to maximize GPU usage

3. Model Size

- Start small, scale up if needed
- DistilBERT: 40% smaller, 60% faster, 97% performance
- Consider model distillation for deployment

4. Hardware

- GPUs with high memory bandwidth (A100, H100)
- Multi-GPU training with data parallelism
- Use FlashAttention when available

2. Architecture Choice:

- When would you use encoder-only vs decoder-only?
- Can GPT do classification? Can BERT generate?
- Why has decoder-only become more popular recently?

3. Scaling:

- Is bigger always better?
- What are the limits to scaling transformers?

- Training transformers: all the components (Lecture 17)

Next week (Week 6):

- Masked Language Modeling
- Pre-training and fine-tuning
- Contextual embeddings in action
- RoBERTa, ALBERT, DistilBERT
- Improvements and optimizations
- Real-world BERT applications
- Cognitive neuroscience connections

- Position-wise transformations
- Add non-linearity and capacity

3. LayerNorm & Residuals

- Critical for training deep networks
- Stabilize gradients, enable deeper models

4. Three Architectures

The original Transformer paper

- **Su et al. (2021)** - "RoFormer: Enhanced Transformer with Rotary Position Embedding" - Modern positional encoding approach
- **Dao et al. (2022)** - "FlashAttention: Fast and Memory-Efficient Exact Attention" - Making transformers faster

Tutorials:

- HuggingFace Course: Chapters 1.4, 1.5
- The Illustrated Transformer:
<https://jalammar.github.io/illustrated-transformer/>

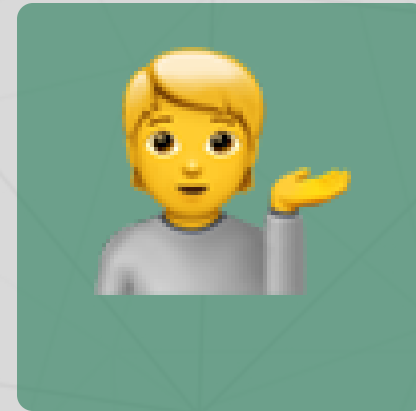
Questions?



Email me



Join our Discord



Come to office hours

Next week: BERT deep dive

Winter 2026